



Pas-à-pas apprenant - Sprint 3 CISIA · InduSense 4.0

À retenir

A quoi sert ce livret ? Il reprend toutes les manipulations vues en classe pour les rejouer pas à pas, à ton rythme. Par module : les actions, les commandes exactes, et [OK] ce que tu dois obtenir. Repo : github.com/thomasfesq/CISIA_24082026_Parcours - Python 3.13, uv. Calendrier 2026 : J1 24/08 · J2 25/08 · J3 26/08 · J4 01/09 · J5 02/09 · J6 03/09. Le formateur annonce en direct le rythme, les pauses et la durée de chaque activité selon l'avancement réel du groupe. Version actualisée le 24/08/2026 : les 12 chargements GitHub et leurs variantes Windows/macOS/Linux sont intégrés à la progression ci-dessous. Comment lire : une puce = une action · un bloc de code = à taper/coller · [OK] Résultat attendu = ce que tu dois voir avant de continuer. Si tu bloques, appelle le formateur. Windows, macOS ou Linux : commence par `[guide_multiplateforme_apprenant_sprint3.md](guide_multiplateforme_apprenant_sprint3.md)`. Il donne, module par module, l'équivalent exact des commandes PowerShell en zsh/bash, les chemins, Docker et le dépannage Linux. Pour les changements de jalon, la procédure Git officielle utilise désormais les scripts courts 01 à 12 indiqués ci-dessous ; la séquence Git manuelle reste uniquement un plan B de diagnostic à utiliser avec le formateur.

Ton terminal - à lire une fois, ça t'évitera la moitié des blocages

Sous Windows, utilisez PowerShell dans le terminal intégré de VS Code (`Terminal > New Terminal`). Sous macOS, utilisez le terminal zsh de VS Code ; sous Linux, utilisez bash. Dans les trois cas, ouvrez le terminal dans le dossier du projet. Les blocs `powershell` sont la référence Windows ; leur équivalent macOS/Linux se trouve dans le guide multiplateforme lié ci-dessus. Sais quelle version tu as (à faire maintenant) :

```
$PSVersionTable.PSVersion
```

► Major = 5 → tu es sur Windows PowerShell 5.1, celui livré avec Windows. Il fonctionne très bien pour tout le sprint, avec une seule réserve : il ne connaît pas l'opérateur `&&`. Si tu colles une commande contenant `&&`, il répond Le jeton « `&&` » n'est pas un séparateur d'instruction valide.

► Major = 7 → tu es sur PowerShell 7, `&&` fonctionne. Rien à faire.

La règle de survie, valable partout : si une commande refuse `&&`, remplace chaque `&&` par un `;`, ou coupe la ligne en deux et lance-les l'une après l'autre. Exemple :

Ce qui peut échouer	Ce qui marche toujours
<code>cd indusense-gameday && git switch -c reparation-x</code>	<code>cd indusense-gameday (entrée) puis git switch -c reparation-x</code>
<code>uv run pytest -q && uv run ruff check .</code>	<code>uv run pytest -q; uv run ruff check .</code>

Attention

Nuance à connaître : `&&` veut dire « seulement si la précédente a réussi », `;` veut dire

« ensuite, quoi qu'il arrive ». Pour un diagnostic (voir toutes les versions), `;` est même préférable.

Pour un enchaînement où l'échec doit stopper la suite (ex. « teste puis publie »), préviens le formateur plutôt que d'improviser.

Astuce

Certains blocs sont marqués ``bash`` (surtout dans les fiches TD) : ce sont des commandes pensées

pour macOS/Linux/WSL2/Git Bash, ou pour un runner de CI. Les commandes `uv`, `git`, `docker`, `pytest` s'écrivent pareil dans les deux mondes ; ce qui diffère, ce sont les enchaînements (`&&`), les chemins (`\` contre `/`) et les commandes propres au shell (`printf`, `export`, `cat`). En cas de doute : demande.



Ta progression GitHub - un jalon au début de chaque demi-journée

À retenir

Dépôt officiel du groupe : https://github.com/thomasfesq/CISIA_24082026_Parcours, main est l'état de départ reconstruit à la fin du Sprint 2. Les branches distantes s'appellent exactement `jalón/01` à `jalón/12`. Attends toujours l'annonce du formateur. Ne fais jamais `git switch jalón/NN` : reste sur ta branche personnelle. Les scripts `mettre_a_niveau` et `verifier_jalón` publiés le 24/08/2026 acceptent le numéro court `NN` et ciblent exactement `jalón/NN`. Un ancien nom long exact reste accepté par compatibilité, mais le numéro court est la notation officielle.

Quand le formateur donne le signal	Ce que tu charges	Première ligne attendue de
J1 24/08, à l'ouverture	clone de main	# Jalón actuel : 00-reconstruction-fin-sprint2
J1 matin, au signal	<code>jalón/01</code> , avant M23	# Jalón actuel : 01-j1-matin-m23
J1 après-midi, au signal	<code>jalón/02</code> , avant M24	# Jalón actuel : 02-j1-apres-midi-m24
J2 matin, au signal	<code>jalón/03</code> , avant M25	# Jalón actuel : 03-j2-matin-m25
J2 après-midi, au signal	<code>jalón/04</code> , avant M26	# Jalón actuel : 04-j2-apres-midi-m26
J3 matin, au signal	<code>jalón/05</code> , avant M27	# Jalón actuel : 05-j3-matin-m27
J3 après-midi, au signal	<code>jalón/06</code> , avant M28	# Jalón actuel : 06-j3-apres-midi-m28
J4 matin, au signal	<code>jalón/07</code> , avant M29 puis M30	# Jalón actuel : 07-j4-matin-m29-m30
J4 après-midi, au signal	<code>jalón/08</code> , avant PayGuard	# Jalón actuel : 08-j4-apres-midi-m31-m32-payguard
J5 matin, au signal	<code>jalón/09</code> , avant InduSense	# Jalón actuel : 09-j5-matin-m31-m32-indusense
J5 après-midi, au signal	<code>jalón/10</code> , avant M33 puis M34	# Jalón actuel : 10-j5-apres-midi-m33-m34
J6 matin, au signal	<code>jalón/11</code> , avant le Game Day	# Jalón actuel : 11-j6-matin-gameday
J6 après-midi, au signal	<code>jalón/12</code> , dans Parcours seulement	# Jalón actuel : 12-j6-apres-midi-retex

Avant chaque jalon : enregistre ton travail, ouvre le panneau Contrôle de code source de VS Code, relis les fichiers modifiés et fais un commit de checkpoint. Ne versionne jamais `.env`, une clé, un token ou un secret. Puis vérifie que `git status --short` ne renvoie rien.

Windows PowerShell - remplace `NN` par le numéro annoncé :

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalón NN
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalón.ps1 -Jalón NN
```

macOS zsh ou Linux bash - remplace `NN` par le numéro annoncé :

```
bash scripts/formation/mettre_a_niveau.sh NN
bash scripts/formation/verifier_jalón.sh NN
```

`mettre_a_niveau` contrôle ta branche et ton dépôt propre, crée une sauvegarde locale, récupère le jalon annoncé et le fusionne sans réécrire l'historique. `verifier_jalón` contrôle ensuite le marqueur, recrée l'environnement avec `uv`, lance les tests puis Ruff : c'est la preuve que ton poste est aligné, pas seulement que Git a fusionné.

À retenir

[OK] Tu peux continuer seulement si la sortie affiche `Jalón intègre sur <ta-branche> : jalón/NN`, puis `Jalón vérifie : jalón/NN`, si la première ligne du marqueur est exactement celle du tableau et si ta branche porte toujours ton nom. Rejouer les scripts est sans danger. En cas de conflit, la fusion est annulée ; appelle le formateur. Il pourra relancer avec `-Rattrapage` sous Windows ou `--rattrapage` sous macOS/Linux. Ton travail reste dans ta branche personnelle et dans `sauvegarde/...` Ne lance jamais `reset --hard`.

Plan B manuel, uniquement si le script est absent ou illisible et avec le formateur :



```
git branch --show-current
git status --short
git fetch origin
git merge --no-edit origin/jalon/NN
git branch --show-current
```

0.1 · Accueil & cap

- Caméras allumées si possible, partage d'écran prêt sur la slide de titre du module 23.

0.2 · Contrôle d'entrée des postes

- Lance la commande de contrôle express correspondant à ton terminal.

Windows PowerShell :

```
python --version; uv --version; git --version; Get-Command code -ErrorAction SilentlyContinue; wsl -l -v; docker --version
```

macOS zsh ou Linux bash :

```
python3 --version; uv --version; git --version; command -v code || true; uname -a; docker --version || true
```

- puis dépose ta capture dans le chat.
- La section « Socle déjà installé » de la note de cadrage (tableau A).

À retenir

[OK] Résultat attendu : Python, uv, Git et VS Code répondent. Sous Windows, WSL doit annoncer VERSION 2 ; cette preuve ne concerne pas macOS/Linux. Seul `docker --version` peut échouer pour l'instant, Docker étant requis avant J3. Pourquoi des ; et pas des && ? Le ; enchaîne les commandes quoi qu'il arrive : même si une ligne échoue, tu vois quand même les suivantes - c'est exactement ce qu'on veut pour un diagnostic. Et il fonctionne dans tous les terminaux (PowerShell 5.1, PowerShell 7, bash). Voir l'encadré « Ton terminal » ci-dessus.

J1 - clone initial et branche personnelle. Dans VS Code, choisis ****Terminal > Nouveau terminal****. Place-toi dans le dossier parent où tu veux conserver le projet, puis colle le bloc correspondant à ton système. Si le dossier existe déjà, ne reclone pas : ouvre-le avec Fichier > Ouvrir le dossier et demande au formateur de vérifier ta branche.

Windows PowerShell :

```
git clone https://github.com/thomasfesq/CISIA_24082026_Parcours.git
cd CISIA_24082026_Parcours
git switch -c prenom-nom
Get-Content -LiteralPath .\FORMATION\JALON_ACTUEL.md -TotalCount 1
git branch --show-current
```

macOS zsh ou Linux bash :

```
git clone https://github.com/thomasfesq/CISIA_24082026_Parcours.git
cd CISIA_24082026_Parcours
git switch -c prenom-nom
head -n 1 FORMATION/JALON_ACTUEL.md
git branch --show-current
```

À retenir

[OK] Résultat attendu : # Jalon actuel : 00-reconstruction-fin-sprint2, puis ta branche personnelle. Remplace réellement `prenom-nom` par ton identifiant, sans espace. Tu ne travailles ni sur `main` ni sur une branche `jalons/*`.

0.3 · Point projet fil rouge

- Au tour de table, présente où tu en es de ton projet fil rouge en fin de Sprint 2.

À retenir

[OK] Résultat attendu : Checklist go/no-go (cf. note de cadrage §5) : model card v1 · contrat I/O · artefacts · seuil · runs MLflow · Gold Dataset. Si l'un manque, tu le combles en autonomie cette semaine.



0.4 · Bascule vers le premier module

- La slide de titre du module 23.

23 - Refactoring & structure projet · J1 matin

[OK] Preuve finale visée : `uv run pytest -q vert` · `uv run ruff check . propre` · `uv run indusense --help` répond (train/predict).

Charge le jalon 01 - seulement au signal du formateur

- Reste sur ta branche personnelle et vérifie que `git status --short` est vide.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 01`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 01`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 01`, puis `bash scripts/formation/verifier_jalon.sh 01`.

À retenir

[OK] Résultat attendu : # Jalon actuel : 01-j1-matin-m23 et ta branche personnelle toujours active. N'anticipe pas jalon/02 : attends l'annonce du formateur.

Préflight & dépannage poste - AVANT la théorie (à faire dès l'ouverture)

- Lance le contrôle express. Sous Windows, `python --version` peut afficher 3.14 : ce n'est pas grave, on cible le venv du projet en 3.13.

À retenir

[OK] Résultat attendu : Ce qui fait foi, c'est `uv run python --version` dans le projet (doit dire 3.13.x). `code --version` et `docker --version` ne sont pas bloquants pour le matin.

Les cas Windows fréquents à régler tout de suite :

```
# 1) uv absent du PATH (cas fréquent) :
winget install --id astral-sh.uv -e          # puis FERMER / ROUVRIRE le terminal
uv --version

# 2) depuis la racine du dépôt déjà cloné :
if (-not (Test-Path -LiteralPath .\pyproject.toml) -or -not (Test-Path -LiteralPath .\uv.lock)) {
    throw "Ouvre le terminal dans la racine CISIA_24082026_Parcours (pyproject.toml et uv.lock attendus)."
}
uv venv --python 3.13
uv sync --frozen --extra dev
uv run python --version          # DOIT afficher Python 3.13.x <-- fait foi
git status --short -- uv.lock    # aucune sortie attendue

# 3) VS Code : commande 'code' introuvable ?
Get-Command code -ErrorAction SilentlyContinue # si rien ne sort :
winget install --id Microsoft.VisualStudioCode -e # puis rouvrir le terminal, ou ouvrir VS Code à la main
```

Alignement local avant M23. Le dépôt `CISIA_24082026_Parcours` contient déjà le socle corrigé utilisé par ce livret. N'applique aucun ancien script `APPLIQUER_CORRECTIF_SPRINT3.ps1` provenant d'un kit antérieur : le marqueur du jalon 01 et les tests ci-dessous sont les preuves qui font foi.

À retenir

Hors ligne. Utilise la copie `CISIA_24082026_Parcours` et le `.venv` préparés la veille. Si le dépôt n'est pas présent, demande la copie locale du kit : ne relance pas un clone au milieu du dossier courant. M23 n'a besoin ni de GPU ni de Docker.

Ouvrir le Sprint 3 - scénario B seulement

- Vérifie tes prérequis Sprint 2, un par un : modèle candidat retenu · model card v1 · contrat I/O JSON · seuils de décision · artefacts modèle · Gold Dataset / données disponibles.
- `note_cadrage_formateur_sprint3` → section « Point projet fil rouge ».



Pourquoi on sort du notebook

- Le contraste à l'écran :

```
# avant - cellule de notebook (logique dupliquée, fragile)
df["temp_lag1"] = df.groupby("machine_id")["temperature"].shift(1)

# après - package (une seule source de vérité, testée)
from indusense.features.temporal import add_temporal_features
df = add_temporal_features(df)
```

Montrer le squelette

- Reste dans le dossier `CISIA_24082026_Parcours` validé au préflight.
- Précisément : `pyproject.toml` (identité, Python ≥ 3.13 , < 3.14 , deps, script `indusense`) · `src/indusense/features/temporal.py` (le `shift(1)` avant le `rolling`) · `src/indusense/data/loaders.py` (`normalize_machine_id`) · `tests/test_temporal.py` (suite anti-fuite, tri temporel et colonne manquante) · `src/indusense/cli.py` (`train` / `predict`).
- À lancer ensemble :

```
uv --version
uv sync --frozen --extra dev
Test-Path -LiteralPath .\pyproject.toml
Test-Path -LiteralPath .\uv.lock
uv run python -c "import indusense; print(indusense.__file__)"
```

À retenir

[OK] Résultat attendu : les deux contrôles renvoient `True` et l'import affiche `CISIA_24082026_Parcours\src\indusense\__init__.py`. Si `uv` est introuvable → cf. Préflight (`winget install --id astral-sh.uv -e`).

Pause obligatoire en visio

- Coupe caméra et partage d'écran, éloigne-toi du poste et reviens au signal du formateur. Ne lance aucune commande pendant la pause.

TP 1 - Structure & pyproject

- Explore `src/indusense/`, `tests/`, `data/`, `artifacts/` et classe ce qui relève de `data` / `features` / `models` / `api` / `cli`.
- `pyproject.toml` bloc par bloc (détail dans la fiche TD 23 / le manuel de révision) : `[build-system]` → `[project]` (nom, version, `requires-python`) → `dependencies` → `[project.optional-dependencies]` `dev` → `[project.scripts]` (`indusense = "indusense.cli:main"`) → `[tool.hatch...]` → `[tool.ruff]` / `[tool.pytest...]`.
- Audite le contrat sans le modifier : `Select-String -Path .\pyproject.toml -Pattern 'requires-python', 'optional-dependencies', 'indusense\s*='`, puis `git diff -- pyproject.toml uv.lock` (aucune sortie attendue).

TP 2 - Anti-leakage

- Ouvre `features/temporal.py`, repère le tri par machine et timestamp, comprends pourquoi `shift(1)` est avant `rolling`, puis complète / rejoue le test anti-fuite.
- `uv run pytest tests/test_temporal.py -q`

À retenir

[OK] Résultat attendu : 0 échec ; ce test prouve la feature décalée. Le split train/test temporel est un mécanisme distinct, traité dans l'exercice avancé.

TP 3 - Normalisation des machines

- Prédit puis exécute :



```
$normalizationCheck = '@'
from indusense.data.loaders import normalize_machine_id as n
ids = ("MACH-01", "MACH_01", "M-06", "M-2")
print([n(raw) for raw in ids])
'@
uv run python -c $normalizationCheck
uv run pytest tests/test_loaders.py -q -k normalize_machine_id
```

► Résultat exact : ['MACH-01', 'MACH-01', 'MACH-06', 'MACH-02'] et 0 échec.

TP 3 · suite et extension facultative

- `normalize_machine_id` dans `loaders.py`.
- (Si le groupe avance) code nouveau à taper - extraire une fonction propre : créer `src/indusense/features/cleaning.py` + `tests/test_cleaning.py` (code fourni dans la fiche TD 23), puis `uv run pytest tests/test_cleaning.py -q`.

Preuve finale

- Tu produis :

```
uv run pytest -q
uv run ruff check .
uv run indusense --help
uv run python --version
```

À retenir

[OK] Résultat attendu : tests à 0 échec (sans compteur imposé) · ruff propre · CLI qui répond (`train/predict`) · Python 3.13.x. Partage tes sorties terminal dans le chat.

À ne pas oublier · FAQ · définitions

À retenir

Définition. Fuite de données : info du futur qui entre dans l'entraînement → score faussement excellent. Package = dossier importable ; module = un fichier `.py` ; refactoring = réorganiser sans changer le comportement. « `src/` layout vs code à la racine ? » → avec `src/`, les tests s'exécutent contre le package installé (comme en prod) → on attrape les erreurs de packaging. « Pourquoi `uv` et pas `pip` ? » → plus rapide et un `uv.lock` (versions exactes) → environnements identiques partout. À ne pas oublier de dire : un score trop beau doit inquiéter ; le découpage temporel n'est pas optionnel.

QCM express + transition

- Réponds aux questions J1 1-3 : package `src/`, rôle de `pyproject.toml`, fuite par split non temporel. La correction attendue est 1-B · 2-C · 3-A, avec justification.
- Supports à garder ouverts : `23_module_refactoring_structure_projet` · `fiches_TD_apprenant_sprint3` · `deck apprenant` publié `01_PRESENTATIONS_PPTX/23_slides_refactoring_structure.pptx`.

24 - CI/CD + tests + versioning · J1 après-midi

[OK] Preuve finale visée : `uv run pytest -q` · `uv run ruff check .` · `uv run black --check .` · `uv run pre-commit run --all-files` · pipeline CI vert · `versioning_strategy.md` · 0 secret.

Charge le jalon 02 - AVANT M24

- Enregistre et committe M23, reste sur ta branche personnelle, puis attends le signal.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 02`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 02`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 02`, puis `bash scripts/formation/verifier_jalon.sh 02`.

À retenir

[OK] Attendu : # Jalon actuel : 02-j1-apres-midi-m24. N'anticipe pas jalon/03 avant l'annonce du formateur en J2.



Reprise

- ▶ À relancer :

```
cd CISIA_24082026_Parcours
uv run pytest -q
uv run ruff check .
git status
```

La CI, le robot qualité

- ▶ `.github/workflows/ci.yml` - insiste sur `python-version: "3.13"`, `uv sync --frozen --extra dev`, `uv run ruff check .`, `uv run black --check .`, `uv run pytest -q`.

Démo PR rouge → verte

- ▶ Si GitHub est prêt : ouvre l'onglet Actions (un job qui passe ou qui échoue). Sinon, en local : casse volontairement un test et observe le log rouge.
- ▶ Corrige, puis `uv run pytest -q` (vert).

pre-commit & secrets

- ▶ `.pre-commit-config.yaml` · `.gitignore`.

Pause obligatoire en visio

- ▶ Coupe caméra et partage d'écran ; reviens à l'heure annoncée. Ne lance pas le hook ni la démonstration de secret pendant la pause.

TP 1 - pre-commit (déjà fourni dans le repo)

```
# pre-commit est DÉJÀ une dépendance dev et .pre-commit-config.yaml est déjà fourni
uv sync --frozen --extra dev
uv run pre-commit install --hook-type pre-commit --hook-type pre-push
uv run pre-commit run --all-files
```

À retenir

[OK] Résultat attendu : ruff corrige tout seul · black reformate · gitleaks bloque un secret.

- ▶ Démo « secret bloqué » réversible : crée une sentinelle explicitement fausse, lance seulement le hook gitleaks (on ne tente pas de vrai commit), puis nettoie. ⚠ Jamais une vraie clé, même révoquée.

```
@'
# Valeurs d'exemple publiques AWS, invalides, réservées à la documentation.
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
'@ | Set-Content -LiteralPath .\fuite_demo.txt -Encoding ascii
git add -f -- .\fuite_demo.txt
uv run pre-commit run gitleaks --files .\fuite_demo.txt
git restore --staged -- .\fuite_demo.txt
Remove-Item -LiteralPath .\fuite_demo.txt
```

À retenir

[OK] Résultat attendu : le hook gitleaks échoue, avec `RuleID: aws-access-token` et `leaks found: 1`. Puis `git status --short` ne montre plus `fuite_demo.txt` : rien n'a été commité, le dépôt est propre.

TP 2 - Workflow GitHub Actions (+ job build)

- ▶ Ouvre `.github/workflows/ci.yml`, identifie les étapes, vérifie Python en 3.13, et ajoute le job `build` (le repo a `quality` mais pas `build`) :



```
build:
  needs: quality
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: astral-sh/setup-uv@v3
    - name: Build wheel
      run: uv build
    - name: Upload artifact
      uses: actions/upload-artifact@v4
      with:
        name: indusense-wheel
        path: dist/*.whl
```

Versioning données - DVC

- En concept ou réel selon le setup (on versionne le gold et le modèle) :

```
# DVC + MLflow sont déjà déclarés dans l'extra `mlops` et verrouillés.
# Ne pas lancer `uv add` : cela réécrirait uv.lock.
uv sync --frozen --extra mlops
# Le starter n'est pas un repo DVC : initialiser avant le premier `dvc add`.
uv run dvc init
# data/gold et rf.joblib sont DÉJÀ suivis par git -> les sortir de l'index AVANT dvc add
git rm -r --cached data/gold/gold_dataset.csv artifacts/models/rf.joblib
uv run dvc add data/gold/gold_dataset.csv artifacts/models/rf.joblib
git status      # 2 pointeurs .dvc + .gitignore mis à jour
uv run dvc status
```

Versioning modèle - metadata / MLflow

- artifacts/models/model_metadata.json.
- Démo MLflow réelle (le registre exige un backend SQLite) :

```
# Remote hors du dépôt, chemin relatif identique sur PC et Mac.
uv run python scripts/demo_versioning.py --remote ../dvc-store
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db # http://localhost:5000
```

- Ils produisent `versioning_strategy.md` (sections Code / Données / Modèle / Secrets - structure fournie dans la fiche TD 24).

Correction, debug et dépôt des preuves

- Rejoue les quatre contrôles de la Definition of Done ; corrige d'abord l'outil qui échoue, puis dépose `versioning_strategy.md` et la sortie de `git status` sans secret.
- [OK] Résultat attendu : tests/lint/format/pre-commit à 0 échec, pointeurs DVC visibles, stratégie lisible et aucun secret dans les fichiers suivis.

QCM de fin de journée

- Passe le QCM J1 (QCM_fin_de_journee_sprint3 / Kahoot kahoot_J1.xlsx), avant la mini-rétro.

À ne pas oublier · FAQ · définitions

À retenir

Définition. CI : serveur qui rejoue lint+tests+build à chaque push. DVC : versionne les gros fichiers (pointeur dans Git + contenu sur un remote). Stage MLflow : candidate → Staging → Production → Archived. « DVC remplace Git ? » → non : Git pour le code, DVC pour les gros fichiers ; ils travaillent ensemble. « `dvc add` dit already tracked by SCM ? » → le fichier est déjà dans git : `git rm -r --cached <fichier>` puis `dvc add`. Et le registre MLflow ne marche qu'avec `mlflow ui --backend-store-uri sqlite:///mlflow.db`. « Pourquoi révoquer un secret au lieu de le supprimer ? » → il reste dans l'historique Git ; exploitable tant qu'il n'est pas révoqué côté fournisseur. À ne pas oublier de dire : reproduire un résultat = code + données + modèle, versionnés ensemble.



Transition → module 25

À retenir

[OK] Résultat attendu : Preuve finale J1 : `uv run pytest -q` · `uv run ruff check .` · `uv run black --check .` · `uv run pre-commit run --all-files`. Livrables : `ci.yml` (+ job build), `.pre-commit-config.yaml`, `versioning_strategy.md`, 0 secret en clair.

25 - API Design & REST (FastAPI) · J2 matin · US3.2 (C7)

[OK] Preuve finale visée : `/docs` lisible · `/health 200` · `/predict-tabular 200` (avec clé) · 401 sans clé · 422 si < 7 relevés · 503 sans modèle · `docs/model_card.md` à trois niveaux, sans preuve inventée.

Charge le jalon 03 - AVANT M25

- ▶ Ouvre `CISIA_24082026_Parcours` dans VS Code, puis Terminal > Nouveau terminal. Vérifie branche personnelle et état propre ; attends le signal.
- ▶ Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 03`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 03`.
- ▶ macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 03`, puis `bash scripts/formation/verifier_jalon.sh 03`.

À retenir

[OK] Attendu : # Jalon actuel : 03-j2-matin-m25. N'anticipe pas `jalons/04` avant l'annonce du formateur.

Reprise, ouverture du poste et préflight

- ▶ Ouvre VS Code, puis Fichier > Ouvrir le dossier... > `CISIA_24082026_Parcours`. Ouvre Terminal > Nouveau terminal et vérifie `uv run python --version`, `Test-Path .\uv.lock` et `git status --short` avant de commencer.

Théorie - l'API comme contrat

- ▶ Slide « Une API, c'est un contrat ».
- ▶ Slide « `/health` vs `/ready` » : `/health` = le process tourne (liveness, 200 dès le lancement) ; `/ready` = le modèle est chargé (readiness, 503 sinon).

Tour du code (l'API du squelette)

- ▶ Précisément, dans `src/indusense/api/` : `schemas.py`, `main.py`, `model_store.py`.
- ▶ Ouvre deux terminaux PowerShell dans VS Code avec Terminal > Nouveau terminal. Le terminal 1 restera occupé par Uvicorn ; les tests se lancent dans le terminal 2. Une variable définie dans l'un n'existe pas dans l'autre.
- ▶ Terminal 1 - serveur :

```
cd CISIA_24082026_Parcours
if (-not (Test-Path -LiteralPath .\env)) {
    Copy-Item -LiteralPath .\env.example -Destination .\env
}
git check-ignore .env # attendu : .env ; ne jamais afficher son contenu
uv run uvicorn indusense.api.main:app --reload
# ouvrir http://127.0.0.1:8000/docs
```

À retenir

[OK] Résultat attendu : `/docs` s'ouvre, les schémas sont visibles, `/health` répond 200.

Pause obligatoire en visio

- ▶ Laisse Uvicorn tourner dans le terminal 1, coupe le partage d'écran et reviens à l'heure annoncée. À la reprise, utilise le terminal 2 pour toutes les preuves.

TP 1 - Endpoints & schémas

- ▶ `payload.json` (un exemple de corps de requête `/predict-tabular`).
- ▶ Depuis `/docs`, exécuter `/predict-tabular` avec l'en-tête X-API-Key (valeur de `.env : INDUSENSE_API_KEY`).

**À retenir**

[OK] Résultat attendu : Réponse 200 avec `proba_panne`, `decision`, `model_version`, `threshold`.

TP 2 - TestClient (les codes d'erreur)

- Dans l'Explorateur Windows, copie le dossier

05_DONNEES_ET_EXERCICES\07_M25_API_PROOFS\tp_api_m25_v1_20260823 du pack apprenant à côté de CISIA_24082026_Parcours. Puis, dans le terminal 2, à la racine du dépôt :

```
$proofRoot = (Resolve-Path -LiteralPath '..\tp_api_m25_v1_20260823').Path
if (-not (Test-Path -LiteralPath (Join-Path $proofRoot 'APPLIQUER_PREUVES_M25.ps1'))) {
    throw 'Ressource M25 absente, incomplète ou placée au mauvais endroit.'
}
& (Join-Path $proofRoot 'APPLIQUER_PREUVES_M25.ps1') -ProjectPath .
if (-not $?) { throw 'Application de la surcouche M25 impossible.' }
uv run pytest -q tests/test_api.py tests/test_readiness_probe.py tests/test_model_card_gate.py
```

À retenir

[OK] Résultat attendu : `M25_OVERLAY=READY`, puis 12 passed, 0 échec ; `/health 200` · sans clé → 401 · historique court → 422 · sans modèle → 503 · requête correcte → 200. Si un fichier différent existait, son chemin de sauvegarde temporaire est affiché ; si tu ouvres un nouveau terminal, redéfinis `$proofRoot`.

À retenir

Le cycle d'une requête `/predict-tabular` - ordre réel du code : authentification puis rate limit (`dependencies=[Depends(require_api_key), Depends(rate_limit_dependency)]`), la validation Pydantic ensuite, le modèle en dernier. Le wrapper `rate_limit_dependency` fixe la politique 60/60 sans exposer `limit` et `window` dans OpenAPI ; le 429 est détaillé au module 26.

Schéma

Le schéma relie les étapes décrites dans cette section ; la version projetée est disponible dans les diapositives du module.

TP 3 - request-id & normalisation au bord

- Activité : ajouter/observer le middleware request-id (déjà présent) ; vérifier que `X-Request-ID` revient dans la réponse.
- (Si le groupe avance) gérer l'endpoint image (`/predict-image`) : renvoyer 422 si le fichier n'est pas une image.

Model Card à trois niveaux (C4/C5)

- Crée `docs/model_card.md` pour trois lecteurs :

1. Métier - finalité, utilisateurs, horizon et seuil réellement mesurés, coût des erreurs, limites et décision humaine.
 2. Technique / maintenance - données/version, features, métriques, dépendances, coût, version du modèle, réévaluation et `run_id` MLflow réel ; sans run, écrire exactement à produire.
 3. Conformité AI Act - usage prévu/interdit, supervision, risques et traçabilité ; écrire « à confirmer avec le référent conformité » tant que la qualification n'est pas établie.
- Le benchmark Marine est un encadré distinct, jamais ton résultat : XGBoost 24 h, seuil $\approx 0,41$, PR-AUC $\approx 0,62$, prévalence 16,6 % ; frugal 202,6 s / 0,158 gCO₂e contre lourd 612,8 s / 0,352 gCO₂e.
 - Plan B hors ligne : produire la carte, marquer `run_id` : à produire et noter l'action de rattachement future ; ne fabriquer jamais un UUID ni une classe AI Act.

Preuve finale

- Tu produis :



```
$proofRoot = (Resolve-Path -LiteralPath '..\tp_api_m25_v1_20260823').Path
uv run pytest -q tests/test_api.py tests/test_readiness_probe.py tests/test_model_card_gate.py
# attendu : 12 passed, 0 échec
$mc = '..\docs\model_card.md'
if (-not (Test-Path -LiteralPath $mc)) { throw 'model_card.md absent' }
$rubriques = @(
    '^## Métier',
    '^## Technique / maintenance',
    '^## Conformité AI Act',
    'run_id'
)
foreach ($rubrique in $rubriques) {
    if (-not (Select-String -LiteralPath $mc -Pattern $rubrique -Quiet)) {
        throw "Rubrique manquante : $rubrique"
    }
}
uv run python .\scripts\validate_model_card.py .\docs\model_card.md --project-root .
if ($LASTEXITCODE -ne 0) { throw 'Model Card non recevable.' }
$lockState = git status --short -- uv.lock
if ($lockState) { throw "uv.lock a changé pendant M25 : $lockState" }
# /docs ouvert + capture : /health 200, 401 sans clé, 422 (<7 relevés)
# le 503 « sans modèle » est prouvé par test_readiness_probe.py
```

À retenir

[OK] Résultat attendu : `/docs` lisible · `/health 200` · 401 sans clé · 422 · 503 sans modèle · 12 tests verts · validateur Model Card sans erreur · `uv.lock` non modifié · trois niveaux présents · `run_id` réel ou à produire · statut AI Act prudent. Partage tes captures et le chemin de la carte.

À ne pas oublier · FAQ · définitions

À retenir

Définition. REST : ressources + verbes HTTP. Liveness (`/health`) : le process tourne. Readiness (`/ready`) : le modèle est chargé. Pydantic : valide et documente le contrat I/O. « Charger le modèle à chaque requête ? » → non, la latence exploserait : une fois au démarrage (`lifespan`). « 401 ou 422 quand la clé manque ? » → 401 (authentification) ; 422 = corps invalide. À ne pas oublier de dire : `/docs` est dérivé du code - la doc ne peut pas mentir.

- « Le contrat I/O du module 22 (Sprint 2) devient les schémas Pydantic ici. »

Transition → module 26

26 - Sécurité & menaces sur l'IA · J2 après-midi · Sécurité (C2)

[OK] Preuve finale visée : `threat_model.md` + `security_controls.md` · 5 contrôles priorités : 4 implémentés et prouvés (401, 422, 429, 413) + audit logging Planifié v0 · suites API/sécurité à 0 échec.

Charge le jalon 04 - AVANT M26

- Dans le terminal Uvicorn, tape `Ctrl+C`. Enregistre et committe M25 sans jamais ajouter `.env`, reviens à la racine du dépôt, puis attends le signal.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 04`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 04`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 04`, puis `bash scripts/formation/verifier_jalon.sh 04`.

À retenir

[OK] Attendu : # Jalon actuel : 04-j2-apres-midi-m26. N'anticipe pas jalon/05 avant l'annonce du formateur en J3.

Ouverture

- Classe les cinq contrôles : auth, validation, rate limit, taille de payload, audit logging.

À retenir

[OK] A trouver : les quatre premiers sont implémentés et prouvables ; l'audit logging est seulement Planifié v0.



Théorie - penser comme un attaquant

- ▶ Slides 6-9 : STRIDE, menaces propres au ML, puis matrice 4 prouvés + 1 planifié.
- ▶ Auth clé → 401 · validation Pydantic v2 → 422 · rate limit 60/min/IP → 429 · payload > 64 Ko → 413.
- ▶ Audit logging → Planifié v0 : preuve future = événement structuré sans clé, payload ni PII + test dédié.

Tour du code (la sécurité du squelette)

- ▶ `src/indusense/api/security.py`, `src/indusense/api/main.py`, les schémas et les deux suites de tests.

```
Select-String -Path .\src\indusense\api\security.py -Pattern 'MAX_BODY_BYTES|def rate_limit|def rate_limit_dependency'  
Select-String -Path .\src\indusense\api\main.py -Pattern 'require_api_key|rate_limit_dependency|limit_body_size'
```

À retenir

[OK] A trouver : politique 60/60 fermée par `rate_limit_dependency`, limite 64 Ko, 401/422/429/413 ; aucun événement ni test d'audit dédié. Un `Content-Length` illisible est refusé par 400.

Pause

- ▶ À la reprise, donne un actif et une menace associée avant de passer au TP.

TP 1 - Attack tree & contrôles testables

- ▶ À la racine de `CISIA_24082026_Parcours`, synchronise exactement l'environnement puis vérifie Python :

```
uv sync --frozen --extra dev  
uv run python --version
```

- ▶ Rédige `threat_model.md` (STRIDE sur `/predict-tabular` + pipeline) et `security_controls.md` avec cinq lignes : statut, preuve actuelle, risque résiduel, action suivante.
- ▶ Dans le registre, écris exactement 4 × Implémenté et 1 × Planifié v0 (audit logging).

À retenir

[OK] Résultat attendu : Python 3.13.x ; fichiers créés sans écraser un travail existant ; aucune clé, aucun payload et aucune PII dans la preuve future d'audit.

TP 2 - Durcir sans casser

```
uv run pytest tests/test_api.py tests/test_security.py -q
```

À retenir

[OK] Résultat attendu : 0 échec ; `/health` reste 200 sans clé ; `/predict-tabular` prouve 401, 422, 429 et 413. Le test direct accepte 60 appels puis bloque le 61e ; une rafale API de 70 appels doit seulement contenir au moins un 429.

TP 3 - preuve complémentaire

- ▶ Au choix : test de l'OpenAPI prouvant que `limit` et `window` ne sont pas exposés, ou test de non-divulgaration dans les logs.

À retenir

[OK] Résultat attendu : preuve verte et intitulé exact. Le test de non-divulgaration ne prouve pas que l'audit logging existe : il reste Planifié v0.



Preuve finale

```
uv run pytest tests/test_api.py tests/test_security.py -q
$rows = @(Get-Content -LiteralPath .\security_controls.md |
    Where-Object {
        $_ -match '^\\ (Auth|Validation|Rate limit|Taille payload|Audit logging) \\|'
    })
$implemented = @($rows | Where-Object {
    $_ -match '\\ Implémenté \\|'
}).Count
$planned = @($rows | Where-Object {
    $_ -match '\\ Planifié v0 \\|'
}).Count
if ($rows.Count -ne 5 -or $implemented -ne 4 -or $planned -ne 1) {
    throw 'Registre attendu : 4 Implémenté + 1 Planifié v0.'
}
```

À retenir

[OK] Résultat attendu : tests à 0 échec ; codes 401/422/429/413 ; cinq lignes de registre correctement classées. Preuve C2 : risques et contrôles ; preuve C8 : tests et risques résiduels.

QCM de fin de JOURNÉE 2

- Passe une seule banque : QCM papier J2 ou Kahoot. Question de contrôle : « lequel reste Planifié v0 ? » → audit logging.

À ne pas oublier · FAQ · définitions

À retenir

Définition. STRIDE : 6 familles de menaces (usurpation, altération, déni d'action, fuite, déni de service, élévation de privilège). Adversarial : entrée truquée pour tromper le modèle. Moindre privilège : n'accorder que le nécessaire. « Un contrôle documenté suffit ? » → non : priorisé ≠ implémenté. Les preuves actuelles sont 401/422/429/413. « Pourquoi laisser /health ouvert ? » → c'est la sonde de liveness de l'orchestrateur ; la protéger casserait le monitoring. « Le request-id est-il un audit log ? » → non : il corréle les requêtes, mais ne constitue ni un événement d'audit structuré ni sa preuve. Plan B hors ligne / sans GPU / sans Docker : tout M26 utilise le code et les tests FastAPI locaux. Si Python ne démarre pas, inspecte code/tests, marque « preuve à rejouer » et ne promeus aucun contrôle. À ne pas oublier : ne jamais logger clé, payload ou PII (log bavard = fuite).

Transition → module 27

- Quatre défenses applicatives sont prouvées, la traçabilité reste planifiée ; le module 27 durcit l'image Docker (multi-stage, non-root, secrets hors image).

27 - Conteneurisation (Dockerfile) · J3 matin · US3.3 (C6)

[OK] Preuve finale visée : docker build OK · docker run → /health 200 → /ready 200 → /predict-tabular 200 · whoami = appuser.

Charge le jalon 05 - AVANT M27

- Lance Docker Desktop, ouvre le dépôt Parcours dans VS Code, vérifie ton checkpoint J2 et attends le signal.
- Windows PowerShell : powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 05, puis powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 05.
- macOS/Linux : bash scripts/formation/mettre_a_niveau.sh 05, puis bash scripts/formation/verifier_jalon.sh 05.

À retenir

[OK] Attendu : # Jalon actuel : 05-j3-matin-m27. N'anticipe pas jalon/06 avant l'annonce du formateur.

Ouverture + préflight Docker

- Préflight obligatoire : docker run --rm hello-world chez chacun (Windows/WSL2).



Théorie - image = couches, multi-stage, non-root

- Slide « Une image = des couches » puis « Multi-stage : builder gros, livrer mince ».

Tour du code (Dockerfile réel)

- Dockerfile du squelette : 2 stages (build avec `uv --frozen --no-dev --no-editable`, runtime `python:3.13-slim`), `useradd appuser (uid 10001)`, `COPY artifacts/models (Variante A)`, `HEALTHCHECK` sur `/health`, `CMD uvicorn ... --host 0.0.0.0 --port 8000`.
- `.dockerignore` (Variante A : exclut tout le lourd mais garde `!artifacts/models`) et le `Makefile` (cibles `sync/test/lint/serve`).

Pause obligatoire en visio

- Coupe le partage et reviens à l'heure annoncée. Docker Desktop reste ouvert, mais aucun build n'est lancé pendant la pause.

TP - build & run

```
cd CISIA_24082026_Parcours
docker build -t indusense:0.1.0 .
docker run -d -p 8000:8000 --name indusense -e INDUSENSE_API_KEY=dev-key indusense:0.1.0
docker exec indusense whoami      # appuser
docker rm -f indusense            # ATTENTION: libère le port 8000 AVANT le module 28
```

À retenir

[OK] Résultat attendu : `/health 200` · `/ready 200` · `/predict-tabular 200` (avec clé) · `docker exec indusense whoami` → `appuser`. Sans le `docker rm -f indusense` final, le `docker compose up` du module 28 échoue sur `port is already allocated`.

TP - durcir & alléger / Variante A

- Vérifier la Variante A : modèle présent dans l'image (sinon `/ready 503`).

Correction guidée et preuve Docker

- Rejoue `docker build`, `docker run`, les trois routes et `docker exec indusense whoami`, puis conserve une preuve lisible de `appuser` et des codes HTTP.
- Termine par `docker rm -f indusense` pour libérer le port 8000 avant M28.

À ne pas oublier · FAQ · définitions

À retenir

Définition. Image Docker : paquet immuable de l'application + son runtime (empilés en couches mises en cache) ; en Variante A, l'artefact modèle y est inclus - l'image n'est PAS « le modèle figé », elle l'emballe. Conteneur : instance en exécution de l'image. Multi-stage : build lourd → runtime mince. Variante A : modèle livré dans l'image → `/ready 200` d'emblée. « gitignoré = absent de l'image ? » → non : Git et Docker sont 2 périmètres distincts (d'où `!artifacts/models`). « Pourquoi non-root ? » → si l'app est compromise, l'attaquant n'est pas root dans le conteneur. À ne pas oublier de dire : dépendances avant le code (cache) et `--host 0.0.0.0` (sinon injoignable).

Transition → module 28

28 - Déploiement local & compose · J3 après-midi · US3.3 (C6)

[OK] Preuve finale visée : `docker compose up -d --wait` → services healthy · smoke tests verts · `/predict 401` sans clé / `200` avec.



Charge le jalon 06 - AVANT M28

- ▶ Arrête le conteneur autonome M27, enregistre et committe le travail du matin, reviens à la racine du dépôt, puis attends le signal.
- ▶ Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 06`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 06`.
- ▶ macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 06`, puis `bash scripts/formation/verifier_jalon.sh 06`.

À retenir

[OK] Attendu : # Jalon actuel : 06-j3-apres-midi-m28. N'anticipe pas jalon/07 avant l'annonce du formateur en J4.

Reprise et préflight Compose

- ▶ Ouvre Docker Desktop, puis dans VS Code Terminal > Nouveau terminal. Reviens à la racine `CISIA_24082026_Parcours`, vérifie `docker version`, `docker compose version` et que le conteneur autonome M27 a bien été retiré.

Théorie - réseau privé & « démarré ≠ prêt »

- ▶ Slide « Plusieurs services qui se parlent » + « Démarré n'est pas prêt ».

Tour du code (docker-compose.yml réel)

- ▶ `docker-compose.yml` : `service api (depends_on: db: condition: service_healthy, env INDUSENSE_API_KEY, INDUSENSE_DB_URL=postgresql+psycopg://...@db:5432/indusense)`, `service db (postgres:16, healthcheck pg_isready -U indusense, volume pgdata)`, `prometheus` + `grafana` déjà déclarés.
- ▶ `.env` (gitignoré) : `INDUSENSE_API_KEY`, `POSTGRES_PASSWORD`. On ne versionne que `.env.example`.

Pause obligatoire en visio

- ▶ Coupe le partage d'écran. À la reprise, reste dans le même terminal et valide d'abord la configuration avant tout up.

TP - compose up & smoke tests

```
if (-not (Test-Path -LiteralPath .\.env)) {
    Copy-Item -LiteralPath .\.env.example -Destination .\.env
}
docker compose config          # DOIT réussir avant tout démarrage
docker compose up -d --wait
docker compose ps              # api + db : healthy ; prometheus + grafana : running
uv run pytest tests/test_smoke_compose.py -q # test À CRÉER au module 28 (sinon smoke via requests)
```

À retenir

[OK] Résultat attendu : `api` + `db` healthy, Prometheus + Grafana running · `/predict-tabular` 401 sans clé, 200 avec · 3 smoke tests verts. Un service sans healthcheck ne peut pas afficher `healthy : running` est alors normal.

Autonomie, correction et preuve du module 28

- ▶ Rejoue le démarrage depuis une stack arrêtée, vérifie le healthcheck de `db`, puis refais les trois smoke tests. Corrige la course au démarrage avant de capturer la preuve finale.

À retenir

[OK] Résultat attendu : `docker compose ps` montre la stack attendue, `/health` répond 200 et le smoke est reproductible. Le module 29 commence au signal du formateur en J4 matin.

QCM de fin de JOURNÉE 3

- ▶ Passe le QCM J3 (`kahoot_J3.xlsx`).



À ne pas oublier · FAQ · définitions

À retenir

Définition. Service : un conteneur déclaré dans compose. Healthcheck : sonde « prêt ? » (≠ « lancé »). Réseau compose : on joint par le nom du service. « localhost ou db ? » → db:5432 ; dans un conteneur, localhost = lui-même. « Pourquoi l'API plante 1 fois sur 2 ? » → course au démarrage : depends_on sans condition: service_healthy. À ne pas oublier de dire : secrets dans .env gitignoré ; on ne versionne que .env.example.

- « L'image du module 27 devient un service dans compose ; le /health du 25 sert de healthcheck. »

À retenir

Topologie compose (noms de service, sens du scrape, emplacement du modèle) :

Schéma

Parcours du schéma : Client → FastAPI non-root → Modèle dans l'image · Variante A → Prometheus · job indusense-api → Grafana

À retenir

(Un 2^e job Prometheus indusense-drift scrape l'exporteur du TP, hors Docker, via host.docker.internal:9109 - down au préflight = normal.)

Transition → module 29

29 - Orchestration Prefect (design) · J4 matin, 1/2 · US3.4 (C6, C7)

[OK] Preuve finale visée : flow « hello » exécutable (retries) · design ingest→feature→predict→store (schéma + table I/O).

Charge le jalon 07 - AVANT M29

- Ouvre le dépôt Parcours dans VS Code, vérifie ta branche personnelle et ton état propre, puis attends le signal.
- Windows PowerShell : powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 07, puis powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 07.
- macOS/Linux : bash scripts/formation/mettre_a_niveau.sh 07, puis bash scripts/formation/verifier_jalon.sh 07.

À retenir

[OK] Attendu : # Jalon actuel : 07-j4-matin-m29-m30. Ce jalon couvre M29 et M30 : ne change rien entre les deux. La référence flows/pipeline.py reste volontairement absente jusqu'au jalon 08. N'anticipe pas ce jalon avant l'annonce du formateur.

Ouverture + préflight (scénario A)

- Préflight : Prefect est déjà dans uv sync (pas de uv add). Vérifier : uv run python -c "import prefect; print(prefect.__version__)" → 3.7.6. Tout tourne en local (pas de serveur Prefect requis).

Théorie ciblée - pourquoi orchestrer

- Slide « Pourquoi orchestrer (et pas juste cron) » + « Idempotence & retries ».

Le piège data - fuite inter-machines (à jouer)

- Debug à jouer : symptôme (pressions incohérentes) → hypothèse en binôme → correction merge_asof(..., by="machine", tolerance="90min", direction="nearest") → test qui repasse.

À retenir

[OK] Résultat attendu : Résidu non-joint ≈ 1,76 %.

TP socle - flow « hello » + design

```
# créer d'abord src/indusense/flows/hello.py + __init__.py vide (code : fiche TD 29)
uv run python -m indusense.flows.hello # pong:indusense + run loggé, retries
```




- Dessiner le flow `ingest→feature→predict→store` + une table I/O (entrées/sorties/erreurs de chaque task), à figer avant d'implémenter.

Preuve, FAQ et correction

À retenir

Définition. Task : étape unitaire. Flow : orchestration des tasks. Idempotent : rejouer ne change rien. Backoff : délai croissant entre réessais. « Pourquoi pas un simple cron ? » → cron lance, point ; un orchestrateur ajoute retries, observabilité, dépendances. « On réessaie toutes les erreurs ? » → non : seulement les transitoires (DB 1 s), pas les déterministes. À ne pas oublier de dire : `merge_asof` sans `by="machine"` ne lève aucune erreur - bug silencieux.

- « Rappelez la fuite de données du module 23 : ici c'est la même idée mais entre machines (`by="machine"`). »

Transition → module 30

Pause obligatoire en visio

- Coupe le partage d'écran et ne dépanne pas pendant la pause. À la reprise, écris dans le chat le backend choisi : PostgreSQL dans Compose ou SQLite sur l'hôte.
- [OK] La preuve d'idempotence est deux runs contre la même base neuve → même nombre de lignes.

30 - Implémentation du flow · J4 matin, 2/2 · US3.4 (C6, C7)

[OK] Preuve finale visée : flow end-to-end · 2 runs → 0 doublon · panne $\approx 4,7802\%$.

Ouverture + préflight (scénario A)

- Préflight : SQLAlchemy + psycopg sont déjà verrouillés. Exécute :

```
uv sync --frozen
if ($LASTEXITCODE -ne 0) { throw "uv sync a échoué : code $LASTEXITCODE" }
$env:PREFECT_PROFILE = "ephemeral"
$env:PREFECT_SERVER_ANALYTICS_ENABLED = "false"
$env:PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY = "false"
$qaStamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$env:COMPOSE_PROJECT_NAME = "cisia_m30_$qaStamp"
docker compose up -d --wait db
if ($LASTEXITCODE -ne 0) { throw "Démarrage DB a échoué : code $LASTEXITCODE" }
```

PostgreSQL @db:5432 exige que le flow tourne dans Compose. Le nom de projet isole sa base des volumes existants ; sur l'hôte, utilise une base SQLite de preuve neuve sous le dossier temporaire Windows (`$env:TEMP`).

Théorie ciblée - historiser & upsert

- Slide « Historiser : la table predictions » + « L'idempotence par upsert ».
- Au bord de store, adapte `prediction_ts` au pilote : `pd.Timestamp(value).isoformat()` pour SQLite ; `pd.Timestamp(value).to_pydatetime()` pour PostgreSQL. Un `pandas.Timestamp` brut n'est pas accepté par SQLite.
- `prediction_ts` conserve l'heure métier sans fuseau, comme les loaders ; seul `created_at` est un instant d'audit avec fuseau. store lit `INDUSENSE_DB_URL` dans son environnement : ne passe ni ne journalise cette URL dans les paramètres Prefect. Ne journalise jamais les colonnes opérateur brutes des incidents.

Le piège data - cible date seule (à jouer)

- Debug : symptôme `panne.mean()` $\approx 4,89\%$ → diagnostic (`time` à minuit) → correction `pd.to_datetime(date.astype(str) + " " + time.astype(str))` → retour à $4,7802\%$.

TP socle - predict & store (upsert) + preuve d'idempotence

- Mode PostgreSQL. Reconstitue l'image après création de `predict_flow.py`, monte le jeu complet en lecture seule et lance le flow dans le service `api` :



```
if (-not $env:INDUSENSE_DATA_DIR) {
    throw "Définir INDUSENSE_DATA_DIR vers le jeu complet"
}
$sourceData = (Resolve-Path -LiteralPath $env:INDUSENSE_DATA_DIR `
    -ErrorAction Stop).Path
docker compose build api
if ($LASTEXITCODE -ne 0) { throw "Build API a échoué : code $LASTEXITCODE" }
$flowArgs = @(
    "compose", "run", "--rm", "--no-deps",
    "-e", "PREFECT_PROFILE=ephemeral",
    "-e", "INDUSENSE_DATA_DIR=/app/data/run",
    "--volume", "${sourceData}:/app/data/run:ro",
    "api", "python", "-m", "indusense.flows.predict_flow"
)
docker @flowArgs
if ($LASTEXITCODE -ne 0) { throw "Flow run 1 a échoué : code $LASTEXITCODE" }
$count1 = (docker compose exec -T db psql -U indusense `
    -d indusense -tA -c "SELECT count(*) FROM predictions;").Trim()
if ($LASTEXITCODE -ne 0) { throw "Lecture count1 a échoué : code $LASTEXITCODE" }
docker @flowArgs
if ($LASTEXITCODE -ne 0) { throw "Flow run 2 a échoué : code $LASTEXITCODE" }
$count2 = (docker compose exec -T db psql -U indusense `
    -d indusense -tA -c "SELECT count(*) FROM predictions;").Trim()
if ($LASTEXITCODE -ne 0) { throw "Lecture count2 a échoué : code $LASTEXITCODE" }
if ([int]$count1 -ne [int]$count2) {
    throw "Idempotence KO : $count1 puis $count2"
}
if ([int]$count2 -ne 15) { throw "Population scorée inattendue : $count2 au lieu de 15" }
```

À retenir

Repli SQLite sur l'hôte. Garde le même jeu complet, sinon tu changes de preuve :

```
if (-not $env:INDUSENSE_DATA_DIR) { throw "Définir INDUSENSE_DATA_DIR vers le jeu complet" }
$env:INDUSENSE_DATA_DIR = (Resolve-Path -LiteralPath $env:INDUSENSE_DATA_DIR `
    -ErrorAction Stop).Path
$env:PREFECT_PROFILE = "ephemeral"
$env:PREFECT_SERVER_ANALYTICS_ENABLED = "false"
$env:PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY = "false"
$qaStamp = (Get-Date -Format "yyyyMMdd_HH:mm:ss") + "_" + ([guid]::NewGuid().ToString("N").Substring(0,8))
$qaDir = New-Item -ItemType Directory -Path (Join-Path $env:TEMP "cisla_m30_qaStamp")
$dbPath = Join-Path $qaDir.FullName "predictions_flow.db"
if (Test-Path -LiteralPath $dbPath) { throw "Refus d'écraser la base : $dbPath" }
$env:INDUSENSE_DB_URL = "sqlite:///$(($dbPath.Replace('\', '/')))"
uv run python -m indusense.flows.predict_flow
if ($LASTEXITCODE -ne 0) { throw "Flow SQLite run 1 a échoué : code $LASTEXITCODE" }
$countScript = '@
import sqlite3, sys
cx = sqlite3.connect(sys.argv[1])
print(cx.execute("select count(*) from predictions").fetchone()[0])
'@
$count1 = uv run python -c $countScript $dbPath
if ($LASTEXITCODE -ne 0) { throw "Lecture count1 a échoué : code $LASTEXITCODE" }
uv run python -m indusense.flows.predict_flow
if ($LASTEXITCODE -ne 0) { throw "Flow SQLite run 2 a échoué : code $LASTEXITCODE" }
$count2 = uv run python -c $countScript $dbPath
if ($LASTEXITCODE -ne 0) { throw "Lecture count2 a échoué : code $LASTEXITCODE" }
if ([int]$count1 -ne [int]$count2) { throw "Idempotence KO : $count1 puis $count2" }
if ([int]$count2 -ne 15) { throw "Population scorée inattendue : $count2 au lieu de 15" }
```

À retenir

[OK] Résultat attendu : les logs résumant `gold_rows=65625, panne_count=3137, panne_rate=0.047802`, sans publier les lignes sources ; `count1 == count2 == 15 · 0` doublon · store a des retries. La base SQLite reste sous `$env:TEMP` et l'ancien `predictions.db` du dépôt n'est jamais ouvert. Le starter donne $\approx 10,4$ % brut / $\approx 10,5$ % entraînable : nomme toujours la population.

Autonomie tutorée courte

- Choisis une seule option : ajouter une assertion anti-doublon dans un test, ou documenter le contrat I/O de `store`. Arrête au signal du formateur même si ce bonus n'est pas fini ; la preuve socle prime.



QCM express 29-30, FAQ et preuve finale

- Passe le QCM J4 (kahoot_J4.xlsx) - premier récap ; la fin de journée 4 (avec le drift PayGuard) arrive après le module 32.

À ne pas oublier · FAQ · définitions

À retenir

Définition. Clé naturelle : colonnes qui identifient une ligne (`machine`, `prediction_ts`). Upsert : insère ou met à jour (`ON CONFLICT ... DO UPDATE`). Idempotence : 2 runs → même `count(*)`. « db est introuvable / Prefect répond 401 Cloud ? » → db n'existe que dans Compose ; utilise le service `api`. Garde `PREFECT_PROFILE=ephemeral` pour la preuve locale. « Pourquoi mes lignes doublent à la relance ? » → `INSERT` simple → passer à l'upsert. « type 'Timestamp' is not supported ? » → le paramètre SQLite est encore un objet pandas ; convertir en ISO 8601 dans `store` sans perdre l'heure. « Pourquoi 4,89 % et pas 4,78 % ? » → cible sur `date` seule (heure à minuit) → recombinaison `date+time`. À ne pas oublier de dire : le flow réutilise la lib testée - il ne réécrit aucune logique métier.

- « Le split temporel du 23 revient : l'horodatage `date+time` décide de la cible panne. »

Transition → module 31

31 - Data drift & métriques (concepts) · J4 après-midi et J5 matin · US3.5 (C3, C8)

Charge le jalon 08 - AVANT PayGuard

- Reviens d'abord à la racine de `CISIA_24082026_Parcours`, enregistre et committe M29-M30, puis attends le signal. Ne lance pas le merge depuis un dossier PayGuard séparé.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 08`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 08`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 08`, puis `bash scripts/formation/verifier_jalon.sh 08`.

À retenir

[OK] Attendu : # Jalon actuel : 08-j4-apres-midi-m31-m32-payguard. Ensuite, lis `FORMATION/EXERCICES/PAYGUARD_README.md`, extrais `FORMATION/EXERCICES/tp_payguard_apprenants.zip` dans un dossier court et séparé, puis ouvre ce nouveau dossier dans VS Code. N'anticipe pas `jalon/09`.

À retenir

Progression partagée M31-M32 : le formateur annonce chaque étape et chaque pause. Passe 1 = TP autonome PayGuard (zip `05_DONNEES_ET_EXERCICES/tp_payguard_apprenants.zip`, définition of done : 12 tests verts). Passe 2 = miroir autonome InduSense livré dans `05_DONNEES_ET_EXERCICES/tp_drift_indusense` : ouvre ce dossier dans VS Code, puis garde `PAS_A_PAS_apprenant_indusense.md` à côté du terminal. Ce miroir possède son propre `.python-version`, `pyproject.toml` et `uv.lock` ; il utilise Python 3.13 / scikit-learn 1.9.0 et ne dépend ni du dépôt GitHub, ni de PayGuard, ni des quatre sources brutes. Dans ce miroir, lance d'abord `uv sync --frozen --extra dev`, puis vérifie `uv run python --version` → Python 3.13.x. La preuve attendue est `uv run python -m pytest .\tests -q` → 11 passed avec le modèle livré ; sans `models\model.joblib`, le résultat intermédiaire normal est 10 passed, 1 skipped jusqu'à `uv run python .\scripts\train_model.py`. Le seuil gelé est 0,05. Les rappels sont F1 0,771 · F2 0,784 · F3 0,053 · janvier 0,728. La fenêtre F2 « capteur +8 °C » donne un PSI 6,845 ; janvier donne 6,213 contre la référence normale mais 0,001 contre la référence haute. Retenir les trois leçons : le PSI hurle sous dérive capteur ; il reste muet face au concept drift F3 malgré le rappel effondré ; une campagne différente exige une référence par régime. ⚠ Ne mélange pas ces mesures réelles avec la simulation synthétique de la fiche TD ci-dessous : `normal(70,4) + 8 °C` donne environ 3,32. Un PSI n'a de sens qu'avec sa population, sa référence et son binning nommés.

Charge le jalon 09 - AVANT InduSense

- Ferme la fenêtre PayGuard, reviens à la racine du dépôt Parcours et attends le signal. Ton dossier PayGuard reste conservé séparément.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 09`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 09`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 09`, puis `bash scripts/formation/verifier_jalon.sh 09`.

**À retenir**

[OK] Attendu : # Jalon actuel : 09-j5-matin-m31-m32-indusense. Ouvre ensuite FORMATION/EXERCICES/tp_drift_indusense comme dossier autonome dans VS Code. N'anticipe pas jalon/10 avant l'annonce du formateur.

J5 matin - séquence exacte dans le terminal du miroir :

```
# 1) Confirmer que VS Code a ouvert tp_drift_indusense, pas son dossier parent.
Test-Path -LiteralPath .\pyproject.toml
Test-Path -LiteralPath .\uv.lock
Test-Path -LiteralPath .\data\reference_normale.csv

# 2) Construire l'environnement sans recalculer le verrou.
uv sync --frozen --extra dev
uv run python --version
uv run python -c "import sklearn; print(sklearn.__version__)"

# 3) Rejouer les quatre fenêtres et nommer la référence à chaque fois.
uv run python .\scripts\drift_lab.py --fenetre 1 --reference normale
uv run python .\scripts\drift_lab.py --fenetre 2 --reference normale
uv run python .\scripts\drift_lab.py --fenetre 3 --reference normale
uv run python .\scripts\drift_lab.py --fenetre janvier --reference normale
uv run python .\scripts\drift_lab.py --fenetre janvier --reference haute

# 4) Produire la preuve M32, puis valider le kit complet.
uv run python .\scripts\alerting_demo.py --report-out .\reports\drift_report_f2.json
uv run python -m pytest .\tests -q -p no:cacheprovider
```

À retenir

[OK] A montrer au formateur : Python 3.13.x, scikit-learn 1.9.0, les deux rapports de janvier (normale puis haute), `sequence=0 -> 1 -> 0`, une seule ligne `drift_events`, le JSON généré et 11 passed. Si `python --version` tout court affiche autre chose, ne l'utilise pas : c'est `uv run python` qui fait foi.

Réserve d'entraînement - ancien protocole intégré/synthétique (optionnel, au signal du formateur)

Le socle J5 est uniquement le miroir et les preuves 6,845 / 0,053 / 11 tests ci-dessus. Si le formateur ouvre ensuite une extension, il peut faire comparer la simulation synthétique `normal(70,4) + 8 °C` (PSI $\approx$ 3,32, KS $p \approx$ 0) ou l'ancien protocole intégré au dépôt fil rouge (PSI 6,834, rappel 0,092, 8 tests). Ces valeurs ne remplacent jamais la preuve officielle : elles illustrent que le PSI dépend de la référence, du binning et de la population.

À retenir

Définition. Covariate drift : $P(X)$ change (entrées). Concept drift : $P(y|X)$ change (relation). PSI : ampleur ($<0,1$ / $0,1-0,25$ / $>0,25$). KS : significativité ($p \approx 0$). « Faut-il des labels pour détecter le drift ? » → covariate : non ; concept : oui. Toujours citer population + référence + binning avec une valeur de PSI.

32 - Rapport de drift + alerting · contrat JSON officiel ; Evidently en extension · US3.5 (C3, C8)

[OK] Preuve officielle J5 : dans le miroir, `alerting_demo.py` génère le rapport JSON, écrit une seule ligne `drift_events` dans SQLite et prouve `sequence=0 -> 1 -> 0`. **Evidently n'est pas installé dans l'environnement du miroir** : un rapport Evidently HTML appartient à la démonstration ou à l'extension du dépôt fil rouge, jamais au préflight obligatoire.

À retenir

Définition. Cooldown : délai mini entre deux alertes. Hystérésis : réarmer seulement quand le PSI repasse sous le seuil. `drift_events` : table qui trace l'alerte. Un bon système d'alerte reste silencieux quand tout va bien.

Extension seulement, sur consigne du formateur : brancher `drift_check` après `predict`, produire un rapport Evidently avec l'extra explicitement installé/verrouillé dans le dépôt fil rouge, puis rejouer sain → 0, dérive → 1, relance → 0. Ne pas lancer cette extension dans le miroir autonome.



33 - Observabilité API (Prometheus) · J5 après-midi, 1/2 · US3.6 (C6, C8)

Charge le jalon 10 - AVANT M33

- ▶ Ferme le terminal du miroir drift ou reviens explicitement à la racine de CISIA_24082026_Parcours. Vérifie branche personnelle, état propre et Docker Desktop prêt ; attends le signal.
- ▶ Windows PowerShell : powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 10, puis powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 10.
- ▶ macOS/Linux : bash scripts/formation/mettre_a_niveau.sh 10, puis bash scripts/formation/verifier_jalon.sh 10.

À retenir

[OK] Attendu : # Jalon actuel : 10-j5-apres-midi-m33-m34. Ce jalon couvre M33 et M34 : aucun changement entre les deux. N'anticipe pas jalon/11 avant le brief J6.

À retenir

Deux cibles à voir UP dans Prometheus (Status → Targets) : indusense-api (l'API du repo) et indusense-drift (lancez `uv run python scripts/export_drift_metrics.py` sur votre poste).

[OK] Preuve finale visée : `/metrics` scrapeable · gauge obligatoire `indusense_model_loaded` visible · 5 SLI/SLO · requêtes PromQL `p95` et `readiness` qui renvoient une valeur.

Ouverture (scénario A)

- ▶ Préflight : l'instrumentator est déjà câblé dans `api/main.py`. Pour isoler réellement les 401, l'instanciation doit être `Instrumentator(should_group_status_codes=False)` avant `.instrument(app).expose(app, endpoint="/metrics", include_in_schema=False)`. Prometheus est déjà dans le compose.
- ▶ Dans VS Code, ouvre le dossier décompressé `PACK_APPRENANTS_SPRINT3_CISIA_20260824`, puis choisis Terminal > Nouveau terminal. Ce premier terminal démarre à la racine du pack ; indique ensuite le chemin du dépôt quand PowerShell le demande :

```
$packRoot = (Resolve-Path -LiteralPath .).Path
$projectInput = Read-Host 'Colle le chemin complet du dossier CISIA_24082026_Parcours'
$projectRoot = (Resolve-Path -LiteralPath $projectInput.Trim('')).Path
$m33Payload = Join-Path $packRoot 'payload.json'
$m33Locustfile = Join-Path $packRoot 'perf\locustfile.py'
$m33VisionLabZip = Join-Path $packRoot '05_DONNEES_ET_EXERCICES\vision_metrics_lab_v1_20260823.zip'
@($projectRoot, $m33Payload, $m33Locustfile, $m33VisionLabZip) | ForEach-Object {
    if (-not (Test-Path -LiteralPath $_)) { throw "Préflight M33 : chemin introuvable : $_" }
}
# Contrôle visuel : si une AUTRE stack tient 3000 ou 9090, appelle le formateur avant de continuer.
docker ps --filter "publish=3000" --filter "publish=9090" --format "{{.Names}} -> {{.Ports}}"
Set-Location -LiteralPath $projectRoot
uv sync --frozen
if ($LASTEXITCODE -ne 0) { throw "uv sync --frozen a échoué (code $LASTEXITCODE)." }
if (-not (Test-Path -LiteralPath .\.env)) { Copy-Item -LiteralPath .\.env.example -Destination .\.env }
docker compose config -q
if ($LASTEXITCODE -ne 0) { throw "Configuration Compose invalide." }
docker compose up -d --wait
if ($LASTEXITCODE -ne 0) { throw "Démarrage de la stack M33 impossible." }
```

À retenir

⚠ Le dépôt contient aussi un `perf\locustfile.py` homonyme : ne l'utilise pas pour M33. Le fichier canonique est celui du pack apprenant, désigné par `$m33Locustfile`.

- ▶ Ouvre maintenant Terminal > Nouveau terminal une seconde fois. Les variables PowerShell ne passent pas d'un terminal à l'autre : redemande donc le chemin du dépôt, lance l'exporteur drift et laisse ce terminal ouvert :

```
$projectInput = Read-Host 'Colle le chemin complet du dossier CISIA_24082026_Parcours'
$projectRoot = (Resolve-Path -LiteralPath $projectInput.Trim('')).Path
Set-Location -LiteralPath $projectRoot
uv run python scripts/export_drift_metrics.py
```

- ▶ Reviens dans le Terminal 1, prouve que l'exporteur répond, puis laisse à Prometheus un intervalle de scrape avant d'ouvrir <http://localhost:9090/targets> :



```
$driftMetrics = curl.exe -fsS http://localhost:9109/metrics
$driftMetricsText = ($driftMetrics -join "`n")
if ($LASTEXITCODE -ne 0 -or $driftMetricsText -notmatch 'indusense_drift') {
    throw "Exporteur drift indisponible ou métrique indusense_drift absente sur le port 9109."
}
$driftMetricsText | Select-String 'indusense_drift'
Start-Sleep -Seconds 15
```

À retenir

[OK] Résultat attendu : `indusense-api` et `indusense-drift` sont toutes les deux UP. Si l'exporteur du Terminal 2 est arrêté, `indusense-drift` repasse DOWN : c'est attendu.

Théorie ciblée - counter/gauge/histogram & cardinalité

- Slide « Mesurer : counter, gauge, histogram » + « Le piège : la cardinalité ».

Tour du câblage (main.py + prometheus.yml)

- `monitoring/prometheus.yml` : `job indusense-api, metrics_path: /metrics, targets: ["api:8000"]`, scrape 15 s.

TP socle - métrique custom + SLO + charge

- Ajouter la métrique custom `indusense_predictions_total` (label `decision`, pas `machine_id`) et la gauge de readiness obligatoire. Dans `api/main.py`, déclarer la gauge au niveau module, la mettre à 1 après le chargement réussi du bundle et à 0 si le chargement échoue ou à l'arrêt :

```
from prometheus_client import Gauge

MODEL_LOADED = Gauge(
    "indusense_model_loaded",
    "1 si le modèle tabulaire est chargé, 0 sinon",
)

# Dans lifespan, avant le try :
MODEL_LOADED.set(0)
# Juste après store._BUNDLE = store.load_bundle(...) :
MODEL_LOADED.set(1)
# Dans except FileNotFoundError, après store._BUNDLE = None :
MODEL_LOADED.set(0)
# Après yield, à l'arrêt :
MODEL_LOADED.set(0)
```

- Puis exécuter les contrôles et la charge depuis `$projectRoot` en conservant les variables du préflight :

```
curl.exe -fsS http://localhost:8000/metrics # répond ; chercher indusense_model_loaded 1
# PromQL : histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
# Readiness instantanée robuste : scrape réussi × modèle chargé.
# Si la gauge est absente, le membre `or` ramène 0 :
# (
#   up{job="indusense-api"}
#   * on(job, instance)
#   indusense_model_loaded{job="indusense-api"}
# )
# or on(job, instance) (0 * up{job="indusense-api"})
# charge (locust HORS lock → outil éphémère épinglé, JAMAIS `uv add`) :
$m33KeyLine = Get-Content -LiteralPath (Join-Path $projectRoot '.env') |
    Where-Object { $_ -match '^\s*INDUSENSE_API_KEY\s*=' } |
    Select-Object -Last 1
if (-not $m33KeyLine) { throw "INDUSENSE_API_KEY absente de .env : interrompre avant Locust." }
$env:INDUSENSE_API_KEY = (($m33KeyLine -split '=', 2)[1]).Trim().Trim(' ').Trim('')
if ([string]::IsNullOrEmpty($env:INDUSENSE_API_KEY) { throw "INDUSENSE_API_KEY est vide." }
uv run --with locust==2.44.4 locust -f $m33Locustfile --headless -u 20 -r 5 -t 30s --host http://localhost:8000
if ($LASTEXITCODE -ne 0) { throw "Locust a échoué (code $LASTEXITCODE)." }
# Version validée localement : locust 2.44.4 sous Python 3.13.14 (22/08/2026).
# Buckets par défaut : 0,1 / 0,5 / 1 s.
# Le p95 PromQL est interpolé : ce n'est pas une mesure fine.
```


**À retenir**

[OK] Résultat attendu : `/metrics` répond · `indusense_model_loaded 1` est visible quand `/ready` vaut 200 · la requête `readiness` vaut 1 (et 0 si scrape ou modèle indisponible) · l'exporteur :9109 répond · 2 cibles UP après 15 s · 5 SLO écrits · la p95 renvoie une valeur.

Preuve, FAQ et correction

À retenir

Définition. Counter (monte), Gauge (monte/descend), Histogram (percentiles, p95). Cardinalité : nb de valeurs distinctes d'un label. SLI (mesuré) vs SLO (objectif chiffré). « Pourquoi `machine_id` interdit en label ? » → explosion de séries (cardinalité) et fuite de donnée. « `/health`, `/ready`, `/metrics` : pareil ? » → non : vivant / prêt / métriques scrapées. « Pourquoi multiplier `up` par `indusense_model_loaded` ? » → `up` seul prouve le scrape, la gauge seule peut rester sur son dernier échantillon ; le produit ne vaut 1 que si la cible est collectée et le modèle chargé. « Comment isoler le taux de 401 ? » → `Instrumentator(should_group_status_codes=False)` - sinon les statuts sont groupés (4xx : 401/404/422 confondus). À ne pas oublier de dire : mesurer sans SLO ne sert à rien - on fixe 5 SLO dès le départ.

- « La fuite du module 26 revient : un `machine_id` en label = donnée sensible exposée (et cardinalité). »

Pause obligatoire en visio

- Dépose la capture `/metrics`, la requête PromQL et `slo.md`, puis coupe le partage d'écran pendant la pause annoncée. Le module 34 commence au signal du formateur.

34 - Dashboards & runbooks (Grafana) · J5 après-midi, 2/2 · US3.6 (C6, C8)

À retenir

Un dashboard « InduSense - dérive & métriques » est déjà provisionné dans Grafana : ouvrez-le, disséquez ses panels (seuils 0,10/0,25), puis relancez `evaluate_drift --fenetre 2` et regardez la jauge PSI basculer - avant de construire VOTRE dashboard SLO.

[OK] Preuve finale visée : deux fichiers distincts - dashboard JSON et export JSON/YAML des règles - plus les deux incidents d'exercice observés en `Firing` et un runbook joué jusqu'au retour `Normal`.

Ouverture

- Reformule la cible : transformer les SLO M33 en panels, créer les règles de production `p95 > 300 ms` et `5xx > 1 %` avec `for: 5m`, puis utiliser deux règles d'exercice déterministes avec `for: 1m`.

Théorie ciblée - du SLO au dashboard, le runbook

- Slide « Du SLO au dashboard » + « Le runbook ». Une alerte = condition + durée ; le `for` : évite le clignotement au moindre pic.

Tour de la stack (Grafana dans le compose)

- `docker-compose.yml` : service grafana (port 3000, `depends_on prometheus`). Datasource = <http://prometheus:9090> (nom de service, pas localhost).

TP socle - dashboard, règles et incidents déterministes

- Construis le dashboard Service / Modèle / Données et exporte son JSON. Dans Grafana Unified Alerting, crée séparément les deux règles de production `p95/5xx` en `for: 5m`, puis les règles d'exercice API morte (`up{job="indusense-api"} < 1`) et pic de 401 avec `for: 1m`.
- Incident A : `docker compose stop api`, observer Pending → Firing, puis `docker compose up -d api` et vérifier `/health` jusqu'au retour `Normal`.
- Incident B : produire pendant plus d'une minute des POST sans clé vers `/predict-tabular`, observer le taux de 401 passer en Firing, arrêter la boucle et vérifier le retour `Normal`.

**À retenir**

[OK] Résultat attendu : dashboard JSON + export séparé des règles · règles de production p95/5xx conservées en `for: 5m` · incidents API-down/401 observés en `Firing` · runbook menant à la résolution. Une latence aléatoire Locust n'est pas une preuve de déclenchement.

Preuve, FAQ et autonomie tutorée

À retenir

Définition. Panel : un graphique (requête PromQL). Alert rule : condition + `for:` + destination. Runbook : symptôme → diagnostic → action → escalade. « Pourquoi `for: 5m` ? » → anti-flap : l'alerte ne se déclenche que si la condition dure. « Grafana affiche 'No data' : pourquoi ? » → datasource = <http://prometheus:9090> (nom de service), pas localhost. À ne pas oublier : un runbook non joué est probablement faux - on le répète comme un exercice incendie.

- Au choix : panel taux de 401, ou runbook drift rédigé puis joué. Dépose les deux exports et les captures `Firing/Normal`.

QCM 33-34 ([kahoot_J6.xlsx](#)) - en fin de J5

- Passe le quiz M33-M34 : son fichier s'appelle `kahoot_J6.xlsx`, mais il se joue bien en fin de J5. Le J6 Game Day n'a aucun classeur Kahoot : son débrief tient lieu de QCM.

Clôture et consigne Game Day

- Note le lien vers les deux exports, la preuve de résolution et le nom du runbook. Le jalon 11 reste fermé jusqu'au brief J6.

JOUR 6 (jeudi 03/09) - GAME DAY « Opération lundi matin »

Charge le jalon 11 - AVANT le brief

- Ouvre le dépôt Parcours, vérifie ta branche personnelle et ton état propre. N'ouvre pas encore le dossier J6 ; attends le signal du formateur.
- Windows PowerShell : `powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 11`, puis `powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 11`.
- macOS/Linux : `bash scripts/formation/mettre_a_niveau.sh 11`, puis `bash scripts/formation/verifier_jalon.sh 11`.

À retenir

[OK] Attendu : # Jalon actuel : `11-j6-matin-gameday` et présence de `FORMATION/EXERCICES/J6/J6-gameday.bundle`. N'anticipe pas `jalón/12` avant l'annonce du formateur.

À retenir

Votre mission : le bundle Git du jalon 11 contient une branche `J6-gameday` volontairement « maintenue » vendredi soir... et plus rien ne marche. Le lanceur crée un clone séparé et une branche `reparation-<binome>`. Vous la ramenez à l'état certifié (tag `v1.0-sain`) : `uv run pytest` vert, chiffres de référence retrouvés, API saine, stack Prometheus/Grafana vivante, dépôt propre - post-mortem + restitution synthétique en fin de journée. Votre support du jour : [FORMATION/EXERCICES/J6/gameday_apprenant.pdf](#) (6 phases guidées, chiffres de référence, questions à se poser, bonnes pratiques, théorie à retenir). Règle d'or : diagnostiqué → corrigé → prouvé → commité.

À retenir

Progression du Game Day : le formateur donne le signal de départ, les pauses et le passage à chaque phase. Suis l'ordre du support sans anticiper la phase suivante.

- Premier geste Windows PowerShell, au signal du formateur et depuis la racine du dépôt Parcours :



```
Write-Host 'Identifiant : lettres, chiffres, point, tiret ou underscore.'
$binome = (Read-Host 'Exemple : equipe01').Trim()
$binomeValide = $binome -match '^[A-Za-z0-9][A-Za-z0-9._-]*$'
if (-not $binomeValide) {
    throw 'Identifiant invalide.'
}
powershell -ExecutionPolicy Bypass -File .\scripts\formation\demarrer_gameday.ps1 -Binome $binome
Set-Location (Join-Path (Split-Path -Parent (git rev-parse --show-toplevel)) "CISIA_J6_GAMEDAY_$binome")
git branch --show-current
git diff --stat v1.0-sain J6-gameday
```

- Premier geste macOS zsh ou Linux bash, au signal du formateur et depuis la racine du dépôt Parcours :

```
printf 'Identifiant du binôme, sans espace (ex. equipe01) : '
read -r binome
bash scripts/formation/demarrer_gameday.sh "$binome"
cd "../CISIA_J6_GAMEDAY_$binome"
git branch --show-current
git diff --stat v1.0-sain J6-gameday
```

- ⚠ On ne travaille JAMAIS directement sur J6-gameday ni main ; les tests se restaurent depuis v1.0-sain, le code se répare en le comprenant. Le remote bundle-local est local et son push est désactivé.
- [OK] Definition of done de la journée : pytest vert (tests identiques à v1.0-sain) · pipeline drift conforme (seuil 0,03 · PSI fenêtre 2 = 6,834 · rappel fenêtre 3 = 0,092) · 2 cibles Prometheus UP · Grafana peuplé · commits locaux propres · post-mortem rendu. Un push n'est fait que si le formateur fournit explicitement un remote de collaboration et sa consigne le jour J.

Charge le jalon 12 dans Parcours - puis reviens au Game Day

- Ne fusionne rien dans le clone CISIA_J6_GAMEDAY_<binome>. Laisse sa branche reparation-<binome> intacte et ouvre un deuxième terminal à la racine de CISIA_24082026_Parcours.
- Windows PowerShell, dans Parcours : powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 12, puis powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 12.
- macOS/Linux, dans Parcours : bash scripts/formation/mettre_a_niveau.sh 12, puis bash scripts/formation/verifier_jalon.sh 12.

À retenir

[OK] Attendu : # Jalon actuel : 12-j6-apres-midi-retex. Lis FORMATION/EXERCICES/J6/reprise_apres_dejeuner.md, puis retourne dans le clone Game Day et vérifie git branch --show-current → reparation-<binome> avant de reprendre la phase 3.

- Bonus (binômes rapides) : dans le miroir tp_drift_indusense, rejouer uv run python .\scripts\evaluate_fenetre.py --fenetre 2 ; attendu F2 : PSI 6,845 et rappel 0,784. Puis lancer uv run python .\scripts\evaluate_fenetre.py --fenetre 3 ; attendu F3 : PSI muet et rappel 0,053. Ne pas confondre ce miroir pédagogique avec la référence Game Day du dépôt fil rouge (6,834 / 0,092).

Débrief Game Day (tient lieu de QCM du jour)

- Pas de QCM « module » aujourd'hui : le débrief sert de récap (qu'est-ce qui a cassé, quelle brique du sprint l'a résolu). Option ludique : 6 questions piochées dans les QCM J1→J5.

Clôture du Sprint 3

- La grille d'évaluation transverse : par apprenant, les preuves accumulées (tests, endpoints, dashboards) → matière pour la soutenance (C1→C9) et le notebook journal de bord.

Débrief, clôture, retex et ouverture Sprint 4

- Partage un point qui a marché et un point à améliorer pendant le retex guidé par le formateur.



Glossaire (annexe - à distribuer si utile)

- ▶ Package / module : dossier importable / fichier `.py`. Refactoring : réorganiser sans changer le comportement. Lockfile : versions exactes figées.
- ▶ Fuite de données : information du futur entrée dans l'entraînement. CI/CD : intégration et livraison continues. DVC / MLflow : versionner données et modèles.
- ▶ REST / endpoint : API HTTP / URL+verbe. Pydantic valide le contrat I/O ; Swagger ([/docs](#)) documente l'API.
- ▶ Liveness / readiness : processus vivant / modèle chargé. lifespan : chargement au démarrage.
- ▶ STRIDE / adversarial : menaces et entrées truquées. Codes clés : 401, 413, 422, 429, 503.
- ▶ Image / conteneur : paquet immuable / instance en cours. Le multi-stage sépare build et runtime ; la Variante A inclut le modèle. Un healthcheck sonde le service ; entre conteneurs, viser `db:5432`, pas `localhost`.